Peer Architecture for sky-db

Status: Implemented (Stages 1–8 complete) **Scope:** Replace the single-writer `Pod` model with a peer-to-peer, append-only transaction log.

---

1. Overview

Instead of a client/server architecture, sky-db moves to a **peer architecture** in which each peer appends transactions to an **append-only log**. On startup a peer builds an index from the transactions it knows of, and rebuilds (incrementally) when it becomes aware of transactions from other peers.

The design rests on four pillars:

1. **iroh-docs** as the sync protocol (a "meta protocol" over `iroh-blobs` + `iroh-gossip`).
2. A **hash chain** (`prev` pointer per transaction) for causality, with a deterministic **linearization** (topological sort + hash tiebreak) for total order.
3. **Query-time merge** (last-writer-wins by linearized order) instead of write-time rejection.
4. **Schema-as-data**: the schema is itself a set of transactions, initialized by the creating peer and appendable/updatable by any peer.

The central invariant: **the trie is append-only and never reorders; only the linearized order changes.** This is what makes "rebuild the index when I learn of a peer's transaction" cheap.

---

2. Motivation

The current model is single-writer. `Pod::transact` (`crates/sky-db/src/objects/pod.rs`) relies on:

- A single global monotonic `Txid` (`max_tx` , `set_max_tx(tx + 1)`).
- A single global `MaxEid` counter for assigning entity ids to temp entities.
- `replace_tail` (cardinality-one) that overwrites the previous value, destroying history.

All three break across peers:

- Two peers can both mint `tx = 5` with different content.
- Two peers can both read `max_eid = 5` and assign `ein = 6` to *different* temp entities.
- `replace_tail` is at odds with an append-only log, because it throws away the history the log is supposed to preserve.

The peer architecture removes the shared counters and moves conflict resolution from write time to query time.

---

3. Protocol choice: iroh-docs

`iroh-docs` is the right layer. It is a "meta protocol" that composes the two things the log needs:

- **`iroh-blobs`** — content-addressed storage (BLAKE3). Each transaction is a blob; peers transfer only the bytes the other side is missing.
- **`iroh-gossip`** — epidemic-broadcast-tree pub/sub. This is the *"rebuild the index when I become aware of new transactions"* trigger: `doc.subscribe()` yields a live stream of new-entry events.

- **`iroh-docs`** itself — a key-value replica whose sync is *range-based set reconciliation* (the Aljoscha Meyer paper). That is the "converge on the same set of entries" primitive, and it handles offline peers / catch-up on startup, not just live gossip.

Mapping: **one replica per database, one entry per transaction.** Because entries are append-only and never overwritten, the last-writer-wins CRDT semantics never fire — each `(author, key)` is written once.

Why not the alternatives

Option	Verdict
Automerge	Not an iroh protocol; a JSON-like CRDT for collaborative editing with rich merge semantics. Wrong shape for an append-only log.
Streaming (raw `iroh-net` streams)	The low-level building block. You'd hand-roll set reconciliation + notification + persistence, reimplementing what docs already does.
`iroh-gossip` alone	Ephemeral; no history, no delivery guarantee to offline peers.
`iroh-blobs` alone	Content-addressed transfer but no "what's new" notification and no set reconciliation.

---

4. Core model

The log entry (transaction)

```
struct Tx {
    prev: Option<Hash>,   // hash of the tip this builds on
    author: AuthorId,     // iroh signer identity
    nonce: [u8; 16],      // random, per transaction
    datoms: Vec<Datom>,   // resolved (concrete eins), including tx metadata datoms
}
// iroh-docs entry key   = BLAKE3(serialize(Tx))   (content-addressed)
// iroh-docs entry value = serialized Tx bytes      (stored as a blob)
```

No global `Txid`, no `MaxEid` counter. The `prev` pointer is the only ordering information a peer asserts.

Linearization (total order)

Every peer derives the same total order from the set of entries:

```
a < b  iff  a is an ancestor of b                (causality: follow prev pointers)
        OR  a,b concurrent and hash(a) < hash(b)  (deterministic fork tiebreak)
```

This is a topological sort with a hash tiebreak. It is a valid total order and a pure function of the log — no negotiation, no rejection. The index of each `Tx` in this order is its **linearized position**.

Merge (query-time, not write-time)

Because every datom is kept (no `replace_tail`), "current value" is resolved when you read, by picking the max-position tx per `(e, a, v)`:

- **Cardinality `One`** — the single value whose latest tx has `dir == In` (last-writer-wins).
- **Cardinality `Many`** — all values whose latest tx has `dir == In` (set union, with add/del resolved by linearized order).

"No rejection" vs. "no conflict"

A hash chain never rejects at the log level — forks are first-class, and every peer's append is always valid. But "no rejection" does not mean "no conflict": concurrent writes to the same datom are **detectable and deterministically resolvable**, not rejected. The cost is that a "successful" write can be semantically overridden by a concurrent one, deterministically. That is the CRDT tradeoff, and it is the right one for a peer architecture.

---

5. Ordering & clocks

The distinction that matters is **causality** (what happened-before what) vs. **total order** (the linear sequence you replay).

Mechanism	What it captures	Total order?	Detects concurrency?
Hash chain (`prev` pointer)	full causal DAG (happens-before)	no (partial order)	yes — forks *are* concurrency
Vector clock	same causal partial order, compactly	no	yes
Lamport clock	logical counter, `max(seen)+1`	yes (with author tiebreak)	no
Wall-clock timestamp	nothing causal	yes, but can *violate* causality	no

- A **hash chain** and a **vector clock** are two representations of the same happens-before relation; the DAG is the explicit version, the vector clock the compact summary.
- A **Lamport clock** is a *logical* counter, not a wall-clock timestamp. Wall clocks skew, so they can stamp causally-related transactions in the wrong order.
- **Concurrency is about ancestry, not distance.** Two transactions are concurrent iff neither is an ancestor of the other. "Same distance from the MRCA" is a proxy, not the criterion — two transactions at *different* distances can still be concurrent.

Content hash vs. linearized position

Each transaction has two identities:

- **Content hash** — stable, globally unique, never changes.
- **Linearized position** — derived, and it *shifts* when a new concurrent transaction with a smaller hash is discovered late.

Key the index on the **content hash** (stable), not the position. If you keyed on position, a late fork would reorder everything and force an index rebuild, breaking the append-only property. The linearized order is a *separate, cheaply-updatable* structure.

---

6. Entity identity

Nonce-derived eins (fully safe)

Temp entities are resolved to concrete eins at transaction-creation time. The fully-safe scheme is nonce-derived:

```
Ein::user(nonce) = hash(nonce) & 0x3FFF_FFFF   // 30-bit (trie key width)
```

Deriving from `prev` alone is **not** enough: two peers forking from the same tip would derive the same ein for the same temp name. The ingredients that guarantee uniqueness:

Ingredient	Disambiguates	Caveat
`prev` (tip hash)	different tips	fails on forks from the same tip
`author`	different peers forking	fails if one author writes concurrently (two devices)
per-author `seq`	every tx from one author	reintroduces a counter (per-author, no coordination)
random `nonce` (128-bit)	everything	needs a random source; nonce stored in the tx

The 30-bit birthday bound

`Ein` is `i32` (non-negative), and trie keys are 30-bit (`TrieKey::MASK = 0x3FFF_FFFF`). Deriving an ein from a hash means truncating to 30 bits, which reintroduces collision risk via the birthday bound: ~50% chance of a collision around 2^15.5 ≈ **46,000 entities**. Fine for small databases, but a real ceiling. Options: accept the bound (detect-and-re-derive on collision), or widen `Ein` to `u64` (ripples through the trie key encoding).

Note the asymmetry with `val_table` : values are already content-addressed (`Vid` derived from `universal_hash::hash(&bytes, 0)` with linear probing), which works because `val_table` is single-writer at insert time. For eins in a peer model you *cannot* probe — two peers can't coordinate on "is this id taken" — so the derivation must be collision-free **by construction**.

---

7. Transaction entities

Every transaction is itself an entity (the Datomic model). Its metadata is ordinary datoms, queryable with the same `find` / `pull` machinery.

What it buys

1. **Unifies tx identity with entity identity.** The "tx" key in the deeper index is just the transaction entity's `Ein` — no separate hash-consing table. `Txid` and `Ein` collapse into one space.
2. **Metadata becomes queryable.** `author` , `prev` , `nonce` , `instant` are attributes on the transaction entity.
3. **The log entry and the entity line up.** The iroh-docs entry key (content hash) identifies the transaction; the transaction entity is its in-db representation.

The clean shape

```
// per transaction, at creation time:
nonce   = random 128-bit
tx_ein  = hash(nonce)                       // nonce-derived, like all other eins
datoms  = user datoms
        + [:db/add tx_ein :tx/author  author]
        + [:db/add tx_ein :tx/prev    prev_hash]
        + [:db/add tx_ein :tx/nonce   nonce]
        + [:db/add tx_ein :tx/instant now]   // optional
content_hash = BLAKE3(serialize(prev, author, nonce, datoms))   // log entry key
```

Two subtleties

1. **The content hash must not be a datom.** Storing `[:db/add tx_ein :tx/hash content_hash]` would make the hash depend on a datom that contains the hash — a self-reference. The content hash stays *external* identity; the entity's `Ein` is nonce-derived, not hash-derived. That is why nonce-derived eins matter: they break the circularity.
2. **The linearized position should not be a datom.** It shifts during convergence (a late fork reorders things), and datoms should be append-only and stable. Keep the order as a *derived* structure, not as datoms.

---

8. Storage layout

Current (packed value)

```
// db_trie.rs (current)
struct Value { id: Txid, dir: Dir }   // packed: 28-bit txid + dir in bit 28
// EAVT: [KEY_EAVT, eid, aid, vid] -> U32(packed tx + dir)   // replace_tail for
card-one
// AEVT: [KEY_AEVT, aid, eid, vid] -> U32(packed tx + dir)
```

`replace_tail` (`insert_deep(..., delete_others=true)` → `InsertOption::DeleteOthers`) clears the subtrie before inserting, destroying history for cardinality-one.

Proposed (deeper, Datomic-style)

Unpack `Value` and push `tx` and `dir` down into the key path, so the datom is a full 5-tuple `[e, a, v, tx, op]` :

```
EAVT: [KEY_EAVT, eid, aid, vid, tx_ein] -> U32(dir)   // no replace_tail; keep every
datom
AEVT: [KEY_AEVT, aid, eid, vid, tx_ein] -> U32(dir)
```

Three things this buys:

1. **The 28-bit txid cap disappears.** `tx` becomes a full-width key component.
2. **History is preserved.** The log keeps every transaction; the index should too.
3. **Conflict resolution moves from write-time to query-time.** Store every datom; resolve "current value" at query time by tx order.

`with_update` (proposed)

```rust
pub(crate) async fn with_update(
    trie: &mut SkyTrieMut,
    aid: Ein,          // already-resolved attribute ein (no attr_map lookup)
    ein: Ein,
    val: Val,
    dir: Dir,
    tx_ein: Ein,       // was txid: &Txid
) -> Result<(), TransactError> {
    let eid = ein.to_i32();
    let vid = val_table::insert(trie, val).await?;
    let eavt_key = [KEY_EAVT, eid, aid.to_i32(), vid.to_id(), tx_ein.to_i32()];
    let aevt_key = [KEY_AEVT, aid.to_i32(), eid, vid.to_id(), tx_ein.to_i32()];
    // no replace_tail — keep every datom; dir is the leaf
    trie.insert_deep(eavt_key, TrieValue::U32(dir as u32), false).await;
    trie.insert_deep(aevt_key, TrieValue::U32(dir as u32), false).await;
    Ok(())
}
```

The `Value { id, dir }` struct and its 28-bit packing are deleted. `set_max_tx` / `KEY_MAX_TXID` are deleted. `schema::save` and `MaxEid` / `EntEid` drop their counter logic.

---

9. Schema

Schema-as-data

The schema is initialized by the peer that creates the repo, but peers can append and update it. `Pod::new(db_spec)` goes away; the creating peer writes a **genesis transaction** (the root, `prev = None`) declaring the initial schema, and every peer derives the schema by replaying the log. `Schema` / `AttrTable` become a *materialized view* of the `:db/ident` + `:db/cardinality` datoms.

Genesis transaction

Built-in attributes are *hardcoded* (fixed eins, always present), so the genesis tx only declares user attributes:

```rust
fn genesis(specs: &[AttrSpec], author: AuthorId) -> Tx {
    let nonce = rand_nonce();
    let tx_ein = Ein::tx(&nonce);
    let mut datoms = Vec::new();

    // declare user attributes (content-addressed eins)
    for spec in specs {
        let ein = Ein::attr(&spec.attr);                // hash(ident)
        datoms.push(datom::add(ein, db::ident(), spec.attr.clone()));
        datoms.push(datom::add(ein, db::cardinality(), spec.cardinality.into()));
    }

    // tx metadata (no :tx/prev — it's the root)
    datoms.push(datom::add(tx_ein, tx::author(), author));
    datoms.push(datom::add(tx_ein, tx::nonce(), nonce));
    datoms.push(datom::add(tx_ein, tx::instant(), now()));
```

```
    Tx { prev: None, author, nonce, datoms }
}
```

The chicken-and-egg is resolved by content-addressing: `Ein::attr("foo") = hash("foo")` is computable with no schema. The built-in attrs (`:db/ident` , `:db/cardinality` , `:tx/*`) are *not* declared in the genesis tx — their eins are constants the system already knows.

Schema struct

```
struct Schema {
    by_ident: HashMap<Attr, Ein>,        // ident -> ein   (for resolve)
    by_ein: HashMap<Ein, Cardinality>,   // ein -> cardinality (for query-time LWW)
}

impl Schema {
    fn empty() -> Self {
        let mut s = Self { by_ident: HashMap::new(), by_ein: HashMap::new() };
        // built-ins: fixed eins, always present
        s.builtin(db::ident(),      Cardinality::One);
        s.builtin(db::cardinality(), Cardinality::One);
        s.builtin(tx::author(),     Cardinality::One);
        s.builtin(tx::prev(),       Cardinality::One);
        s.builtin(tx::nonce(),      Cardinality::One);
        s.builtin(tx::instant(),    Cardinality::One);
        s
    }

    fn builtin(&mut self, ident: Attr, card: Cardinality) {
        let ein = Ein::attr(&ident);          // hash(ident), a constant
        self.by_ident.insert(ident, ein);
        self.by_ein.insert(ein, card);
    }

    fn resolve(&self, ident: &Attr) -> Option<Ein> {
        self.by_ident.get(ident).copied()
    }

    fn apply(&mut self, datoms: &[Datom]) {
        for d in datoms {
            if d.dir != Dir::In { continue; }
            let ein = d.ent.to_eid();
            if d.attr == db::ident() {
                if let Dat::Val(Val::String(ident)) = &d.dat {
                    self.by_ident.insert(Attr::from(ident.clone()), ein);
                }
            } else if d.attr == db::cardinality() {
                if let Dat::Val(v) = &d.dat {
                    self.by_ein.insert(ein, Cardinality::from(v.clone()));
                }
            }
```

```
        }
    }
}
```

`apply` recognizes schema declarations by *ident* (`"db/ident"` , `"db/cardinality"`), which is safe because those are built-in and always resolvable. It only folds `Dir::In` datoms — a `:db/ident` retraction would be a schema *deletion*, which is deferred (see §12).

Malformed vs. conflicting

Schema-as-data reintroduces a *kind* of rejection, distinct from the LWW merge:

- **Conflicting** transactions (two peers write the same datom concurrently) → resolved by LWW, never rejected.
- **Malformed** transactions (reference an ident not declared earlier in the linearized order) → *deterministically invalid*, skipped by every peer.

The second case is the "schema race": peer A references `:foo` while peer B concurrently declares `:foo` . In the linearized order, one comes first. This is **deterministic** — every peer computes the same order and reaches the same verdict — so it is not a coordination failure.

---

10. Value widening

`Val` must grow to support the tx metadata (`:tx/author` = 32-byte key, `:tx/prev` = 32-byte hash, `:tx/nonce` = 16 bytes, `:tx/instant` = u64) and to make entity references first-class.

The widened `Val`

```
pub enum Val {
    U32(u32),
    U64(u64),
    String(String),
    Bytes(Vec<u8>),
    Ein(Ein),
}
```

Byte encoding

Extend the existing type-byte scheme (`val_table.rs`):

```
const VAL_TYPE_U32: u8 = 16;
const VAL_TYPE_STRING: u8 = 17;
const VAL_TYPE_U64: u8 = 18;
const VAL_TYPE_BYTES: u8 = 19;
const VAL_TYPE_EIN: u8 = 20;

fn bytes_from_val(val: &Val) -> Vec<u8> {
    match val {
        Val::U32(u)    => { let mut b = vec![VAL_TYPE_U32];
b.extend_from_slice(&u.to_be_bytes()); b }
        Val::U64(u)    => { let mut b = vec![VAL_TYPE_U64];
```

```
b.extend_from_slice(&u.to_be_bytes()); b }
        Val::String(s) => { let mut b = vec![VAL_TYPE_STRING];
b.extend_from_slice(s.as_bytes()); b }
        Val::Bytes(bs) => { let mut b = vec![VAL_TYPE_BYTES];
b.extend_from_slice(bs); b }
        Val::Ein(e)    => { let mut b = vec![VAL_TYPE_EIN];
b.extend_from_slice(&e.to_i32().to_be_bytes()); b }
    }
}

fn val_from_bytes(bytes: &[u8]) -> Val {
    match bytes[0] {
        VAL_TYPE_U32    =>
Val::U32(u32::from_be_bytes(bytes[1..5].try_into().unwrap())),
        VAL_TYPE_U64    =>
Val::U64(u64::from_be_bytes(bytes[1..9].try_into().unwrap())),
        VAL_TYPE_STRING =>
Val::String(String::from_utf8(bytes[1..].to_vec()).unwrap()),
        VAL_TYPE_BYTES  => Val::Bytes(bytes[1..].to_vec()),
        VAL_TYPE_EIN    =>
Val::Ein(Ein::from(i32::from_be_bytes(bytes[1..5].try_into().unwrap()))),
        _ => unreachable!(),
    }
}
```

The key invariant: **the encoding must be injective** — distinct `Val`s must produce distinct byte sequences, or the hash-consing table would collapse two values into one `Vid`. The type byte guarantees this. `val_table::insert` / `query` are unchanged; the `Vid` derivation (`universal_hash::hash(&bytes, 0) & TrieKey::MASK`) is agnostic to the payload.

Ripples

```
// val.rs
impl From<Ein> for Val {
    fn from(e: Ein) -> Self { Val::Ein(e) }        // was Val::U32(e.to_i32() as
u32)
}
impl From<u64> for Val { fn from(u: u64) -> Self { Val::U64(u) } }
impl From<Vec<u8>> for Val { fn from(b: Vec<u8>) -> Self { Val::Bytes(b) } }
impl From<&[u8]> for Val { fn from(b: &[u8]) -> Self { Val::Bytes(b.to_vec()) } }

// pod.rs — Dat::Ent resolution
Dat::Ent(ent) => {
    let eid = /* resolve temp or id */;
    Val::Ein(eid)                                   // was val(eid) -> Val::U32
}

// ein.rs
impl From<Val> for Ein {
    fn from(val: Val) -> Self {
        match val {
```

```
            Val::Ein(e) => e,
            Val::U32(v) => Ein(v as i32),           // legacy path, can drop in a
greenfield rewrite
            _ => panic!("not an ein"),
        }
    }
}
```

`Cardinality::from(Val)` is unchanged (`:db/cardinality` stays `U32(0)` / `U32(1)`). The `Val` accessor helpers (`i32()` / `u32()` / `as_str()`) need `u64()` / `bytes()` / `ein()` siblings.

The query-layer split

`ValsInSlot` and friends match on `Val` equality, so a query for `Val::U32(42)` will *not* match a stored `Val::Ein(Ein(42))` . That is correct (they are different things now), but the `find` API needs a way to express "entity reference" vs. "integer" — likely a `Val::Ein` constructor in the query DSL. This is the one place the widening is *not* purely mechanical.

---

11. Replay algorithm

The central invariant: **the trie is append-only and never reorders; only the linearized order changes.** Replay is two separable halves — insert datoms (order-independent) and compute the order (the only part that shifts on a late fork).

```
struct Index {
    trie: SkyTrie,                 // append-only datom storage
    order: HashMap<Ein, u32>,      // tx_ein -> linearized position (the only
mutable part)
}

impl Index {
    async fn replay(&mut self, txs: &HashMap<Hash, Tx>) -> Result<(), TransactError>
{
        // 1. Linearize: topological sort + hash tiebreak.
        let order = linearize(txs);
        self.order = order.iter().enumerate()
            .map(|(i, h)| (Ein::tx(&txs[h].nonce), i as u32))
            .collect();

        // 2. Insert every datom (append-only; order-independent).
        self.trie.edit(async |trie| {
            for tx in txs.values() {
                let tx_ein = Ein::tx(&tx.nonce);
                for datom in &tx.datoms {
                    with_update(trie, aid, datom.ent.to_eid(), datom.dat.clone(),
                                datom.dir, tx_ein).await?;
                }
            }
            Ok(())
        }).await?;
        Ok(())
```

```
    }
}
```

Linearize

Each `Tx` has one `prev`, so the log is a forest (each node one parent, many children). Topological sort with a min-heap keyed by hash gives a deterministic total order:

```
fn linearize(txs: &HashMap<Hash, Tx>) -> Vec<Hash> {
    // children[prev] = hashes of txs that point at `prev`
    let mut children: HashMap<Option<Hash>, Vec<Hash>> = HashMap::new();
    for (hash, tx) in txs {
        children.entry(tx.prev).or_default().push(*hash);
    }

    // min-heap of ready txs, keyed by hash (deterministic tiebreak for forks)
    let mut ready: BinaryHeap<Reverse<Hash>> = BinaryHeap::new();
    for root in children.get(&None).into_iter().flatten() {
        ready.push(Reverse(*root));
    }

    let mut order = Vec::with_capacity(txs.len());
    while let Some(Reverse(hash)) = ready.pop() {
        order.push(hash);
        if let Some(kids) = children.get(&Some(hash)) {
            for kid in kids { ready.push(Reverse(*kid)); }
        }
    }
    debug_assert_eq!(order.len(), txs.len());   // unreachable: cycles are
impossible
    order
}
```

Two properties:

- **Cycles are impossible by construction.** A tx's hash covers its `prev`, so a cycle would require a hash to contain itself. Content-addressing gives a DAG for free.
- **The tiebreak is total and stable.** "Smallest hash" is a total order over BLAKE3 hashes, and a tx's hash never changes, so every peer computes the identical order.

The merge lives at query time

```
fn current_fills(&self, ein: Ein, attr_ein: AttrEin) -> Vec<Vid> {
    let mut latest: HashMap<Vid, (u32, Dir)> = HashMap::new();
    for (vid, tx_ein, dir) in self.all_entries(ein, attr_ein) {
        let pos = self.order[&tx_ein];
        match latest.entry(vid) {
            Entry::Vacant(e) => { e.insert((pos, dir)); }
            Entry::Occupied(mut e) => {
                if pos > e.get().0 { *e.get_mut() = (pos, dir); }
            }
```

```
        }
    }
    latest.into_iter()
        .filter(|(_, (_, dir))| *dir == Dir::In)
        .map(|(vid, _)| vid)
        .collect()
}
```

Cardinality-one is "the single vid whose latest tx has `dir == In`"; cardinality-many is "all such vids." This is the LWW-by-linearized-order merge, a pure function of `(trie, order)`.

Running-schema replay

Because the schema is data, replay builds it incrementally, in linearized order. The ordering inside the loop is the crux — a transaction resolves against the schema *before* its own declarations are folded in, so "declare before use" holds across transactions, not within one:

```
async fn replay(&mut self, txs: &HashMap<Hash, Tx>) -> Result<(), TransactError> {
    let order = linearize(txs);
    self.order = /* tx_ein -> position */;

    let mut schema = Schema::empty();

    self.trie.edit(async |trie| {
        for hash in &order {
            let tx = &txs[hash];
            let tx_ein = Ein::tx(&tx.nonce);

            // 1. resolve + insert every datom against the *current* schema
            for datom in &tx.datoms {
                let Some(aid) = schema.resolve(&datom.attr) else {
                    continue;                // malformed: undeclared ident, skip
                };
                with_update(trie, aid, datom.ent.to_eid(), datom.dat.clone(),
                            datom.dir, tx_ein).await?;
            }

            // 2. fold this tx's schema declarations into the running schema
            schema.apply(&tx.datoms);
        }
        Ok(())
    }).await?;
    Ok(())
}
```

The `continue` on unresolved idents is the "deterministically skip malformed transactions" policy, applied at the *datom* level. To skip the whole transaction atomically, resolve all idents first and bail on the first miss — a policy choice, not a structural one.

Incremental update

The full `replay` above is the startup path. For a live arrival:

- **New tx builds on the current tip** (common case): append its datoms, append it to the end of `order` — O(1) to the order.
- **New tx is a fork** (its `prev` is an earlier tx): insert its datoms, then re-linearize — the order shifts, but the trie does *not*. Only `self.order` is recomputed (O(n log n), rare).

So "rebuild the index" is really "append datoms + recompute the order map." The expensive part (the trie) is never rewritten.

Caveat: a fork must re-resolve against the historical schema. A fork's linearized position is in the *middle* of the order, so its datoms must be resolved against the schema *as of that position*, not the current (tip) schema. If a late fork references an ident declared after its position, a naive incremental apply (resolving against the tip schema) would insert a datom that a fresh full replay would have skipped as malformed — a divergence. Correct incremental handling therefore requires either re-resolving the fork's datoms against the schema at its position (a history of schema states), or falling back to a full replay when a fork arrives. The "append-only trie" claim holds for the *common* case (a tx building on the tip); the fork case is where the incremental path gets subtle.

---

12. Deferred / open questions

1. **System/user distinction.** Reserve a bit of the `Ein` (e.g. bit 29) as a system flag, or use a marker attribute. Trivially addable later; it is a filter predicate in `list_entities` / `find`. Tradeoff: a bit flag drops user space to 29 bits (~23k birthday bound); a marker attribute costs a query per entity.
2. **`Ein` as a first-class value.** Orthogonal to replay; a type-safety/disambiguation improvement. Partially covered by `Val::Ein` in §10, but the `find` /query DSL changes are deferred.
3. **`find` /query-layer changes** for the `Ein` / `U32` split.
4. **Schema deletion policy.** `apply` only folds `Dir::In`, so the schema is effectively append-only. Attribute deletion is genuinely hard in a replicated log (it orphans existing datoms); "deprecate, don't delete" is the likely answer.
5. **`Ein` width.** 30-bit birthday bound (~46k entities). Widen to `u64` if needed; ripples through the trie key encoding.
6. **`tip()` semantics.** The tip is `order`'s last element, used as the next `prev`. A peer's tip is "the max of what I've seen so far" — it advances as new txs arrive, which is fine because `prev` only records what the peer knew at creation time.
7. **Incremental replay.** The §11 incremental path (append-only trie, re-linearize on forks) is a deferred optimization with no assigned stage — Stage 8 uses full replay on each sync event. The fork case's schema re-resolution caveat (§11) is the main reason it is not the first cut.

---

13. Code impact

The implemented layout (Stages 1–8):

File	Role
`objects/pod.rs`	`Pod` — an immutable snapshot (schema + trie + order), query-only.
`objects/peer.rs`	`Peer` — owns the log + `Arc<RwLock<Index>>`; `transact`/`ticket`/`snapshot`; background replay loop.
`peer.rs`	`create`/`load`/`join` entry points, returning a `Peer`.

`_internal/services/log.rs`	`Log` — iroh-docs wrapper (endpoint + blobs + gossip + docs + router); `append`/`entries`/`ticket`/`subscribe`.
`_internal/services/replay.rs`	`Index`, `apply_tx`, and `replay` (linearize + insert + fold schema).
`_internal/services/db_trie.rs`	`with_update` takes `aid` + `tx_ein`; history-preserving index.
`_internal/services/val_table.rs`	`U64`/`Bytes`/`Ein` byte encoding.
`types/val.rs`	`U64`/`Bytes`/`Ein` variants.
`types/ein.rs`	nonce-derived `Ein::user`/`tx`/`attr`.
`types/tx.rs`	`Tx` (`prev`/`author`/`nonce`/`datoms`) + `linearize`.
`types/schema/*`	`Schema`/`AttrTable` as a materialized view; built-in `:tx/*` attrs.
`traits.rs`	`Transact` on `Peer`; `DbQuery` on `Pod`.

Dependencies

- **`getrandom`** for nonces.
- **`blake3`** for content hashes (iroh-blobs already uses it) — distinct from `universal_hash`, which stays a 30-bit trie-key hasher.
- **`iroh-docs`** (and transitively `iroh-blobs`, `iroh-gossip`) for the log.

---

14. Implementation stages

The work is broken into two phases. **Phase A** is a set of refactors that keep the system single-writer and behaviorally unchanged, each testable against the existing suite. **Phase B** adds the peer model itself. Each stage compiles and passes tests on its own.

Phase A — Refactors (single-writer, no behavior change)

Stage 1 — Widen the value type. Add `U64` and `Bytes` to `Val` (and, optionally, `Ein`), and teach the value table to encode/decode them. This is a pure extension — existing `U32` / `String` values are untouched. *Done when:* the value table round-trips every variant and all existing tests still pass.

Stage 2 — Switch to a history-preserving index. Replace the packed `txid + dir` leaf with a deeper `[entity, attr, value, tx] → dir` layout, and stop overwriting on cardinality-one. "Current value" becomes a query-time computation (latest tx wins) instead of a write-time overwrite. *Done when:* existing tests return identical results, and new tests show history is retained.

Stage 3 — Derive entity ids from nonces. Stop allocating entity ids from a shared counter; mint a random nonce per temp entity and derive its id from it. *Done when:* temp entities get unique, stable ids with no counter, and existing tests pass.

Phase B — Peer model (new capabilities)

Stage 4 — Make every transaction an entity. Give each transaction a nonce-derived id and metadata attributes (`:tx/nonce`, `:tx/instant`; the `:tx/author` and `:tx/prev` values, plus the content hash, wait for the log in Stages 6–7). The monotonic `Txid` remains the index key for now. *Done when:* each transaction produces a queryable entity carrying its own metadata.

Stage 5 — Treat the schema as data. Replace the fixed, up-front schema with a genesis transaction that declares the initial attributes, and derive the schema by replaying the log. *Done when:* the schema is declared via a transaction and reconstructed from the log, with "declare before use" enforced.

Stage 6 — Linearize and replay. Add the hash chain (`prev` pointer), the deterministic linearization (topological sort + hash tiebreak), and the replay loop that inserts datoms and computes the order. This is also where the index key swaps from the monotonic `Txid` to the transaction entity's nonce-derived `Ein` — the linearized order supplies the ordering that "current value" resolution needs. *Done when:* concurrent transactions merge deterministically and every peer computes the same order.

Stage 7 — Wire up iroh-docs. Add the actual log: append transactions to an iroh-docs replica, and sync on startup (one-shot). Add the `create` / `load` / `join` entry points. *Done when:* two peers sync and converge on the same index.

Stage 8 — Live updates and stable snapshots. Add a `Peer` object that owns the log, subscribes to incoming transactions, and replays them automatically in the background; the `Pod` becomes an immutable snapshot handed out on demand. `create` / `load` / `join` return a `Peer` ; `transact` and `ticket()` move to `Peer` . *Done when:* a peer's index updates automatically as peers transact, and the user can hold a stable `Pod` snapshot while querying.

Why this order

- **Stage 2 is the hard prerequisite, done first in isolation.** The existing `replace_tail` is fundamentally incompatible with the peer model (it destroys history, and LWW-by-insertion-order can't handle concurrent writes). Doing it while still single-writer (monotonic `Txid` as the tx key, "max `Txid` " as the merge) isolates the riskiest storage change from the riskiest algorithm change.
- **Stages 1 and 3 are independent of Stage 2** and of each other, so they can be done in any order — or in parallel if the work is split across contributors.
- **Stage 4 introduces the transaction entity** (nonce-derived id + `:tx/*` metadata) but keeps the monotonic `Txid` as the index key. The key swap to the transaction entity's `Ein` is coupled to Stage 6, because "current value" resolution needs an ordering — the monotonic `Txid` provides it today, and the linearized order replaces it in Stage 6.
- **Stages 5 and 6 are orthogonal** (schema evolution vs. ordering) and both sit on Stage 4, so they can be done in either order once 4 lands.
- **Stage 7 adds the log** (iroh-docs) with one-shot sync — the only stage with an external dependency (iroh) and networking. It lands the `create` / `load` / `join` entry points and the append-and-replay path.
- **Stage 8 adds live updates** — a `Peer` object that subscribes and replays automatically, with the `Pod` as a stable snapshot. Deferred because it is a separate concern (live sync vs. one-shot sync) and builds on Stage 7's log.

Appendix: key invariants

- The trie is append-only; only the linearized order changes.
- Content hash is external identity; never a datom (self-reference).
- Linearized position is derived; never a datom (unstable during convergence).
- Ein derivation is collision-free by construction (nonce/ident), not by probing.
- `Val` encoding is injective (type byte disambiguates).
- Cycles are impossible by construction (content-addressed `prev`).
- Schema is append-only (deletion deferred); "declare before use" across transactions.